

qala

Solution Factory Operating System

Platform Design Document

Version 1.0
March 2026

CONFIDENTIAL

INTERNAL USE ONLY

DRAFT v1.0

1. Executive Summary

qala is a Solution Factory Operating System (SFOS) — a unified, hierarchical platform for designing, building, managing, and operating solutions of every type, at any scale. From a solo developer organizing a personal project to a global enterprise managing thousands of products and services, qala provides a single standardized interface and model for the full solution lifecycle.

The platform is itself a solution factory — the root factory — capable of producing tiered, hierarchical solution factories, development environments, and models. qala treats every deliverable as a typed solution with a consistent structure, governed lifecycle, and traceable value chain.

Core principle: every solution, regardless of type or scale, is created, maintained, and operated within a standardized, reproducible, and governed environment — the Solution Development Environment (SDE).

1.1 Platform Objectives

qala is designed to achieve the following outcomes:

- Standardize the creation and management of all solution types (applications, systems, goods, products, services, platforms) under a single operating model
- Enable reproducible, hermetically sealed development and build environments at every tier of the factory hierarchy
- Provide end-to-end lifecycle management from ideation through deployment, operation, and archival
- Integrate CI/CD, configuration management, security, testing, benchmarking, and observability into a coherent, automated system
- Scale seamlessly from individual developers to enterprise organizations without changing the underlying model
- Embed AI-driven intelligence throughout — in optimization, prediction, anomaly detection, and recommendation

1.2 Scope

This document covers the full architectural design of the qala platform including:

- Solution Model: types, structures, and data
- Solution Development Environment (SDE): design, provisioning, lifecycle, and governance
- Solution Factory (SF): orchestration of networked SDEs
- Hermetic Build System and CI/CD pipeline

- Testing, benchmarking, and quality management
- Security, privacy, and threat management
- Release, deployment, and artifact management
- Governance, standards, and compliance
- Data platform, analytics, and AI integration
- API surface and microservice architecture

2. Solution Model

The qala Solution Model defines a universal structure for representing any type of solution. All solutions share the same compositional grammar, data type system, and interface semantics, regardless of their domain or complexity.

2.1 Solution Types

qala recognizes six canonical solution types. Every artifact managed in the platform is classified as one of these types:

Type	Description
Application	A software application with defined processes and user interactions
System	A composition of applications with coordinated behavior and shared infrastructure
Good	A tangible or digital deliverable produced as an output of a process
Product	A market-facing offering combining software, services, and/or goods
Service	A capability or function delivered to a consumer on-demand or continuously
Platform	A substrate on which other solutions are built, deployed, and operated

2.2 Solution Structure Model

Every solution is composed through a nested containment hierarchy. This model is consistent across all solution types, enabling uniform tooling, governance, and analysis.

System → Application → Process → Component → Interface → Message → Data Structure → Data

Layer	Contains	Notes
System	Applications	Top-level organizational boundary
Application	Processes	Executable units with defined behavior
Process	Components	Logical groupings of functionality
Component	Interfaces	Implementation units with explicit contracts
Interface	Messages + Imports/Exports	Boundary layer; defines inbound/outbound contracts

Layer	Contains	Notes
Message	Data Structures	Either Event (dynamic) or State (static)
Data Structure	Data fields	Typed fields using the qala data type system

2.3 Data Type System

qala defines a canonical set of data types used across all solution data structures. Custom types may be composed from these primitives.

bool	string	int	float	double	null
varchar	char	date	array	tuple	set
map	object	pointer	custom		

2.4 Solution Registry

All solutions created in qala are registered in the Solution Models Registry. The registry provides:

- A global catalog of all solutions, their types, versions, and lifecycle states
- Lineage tracking: which solutions depend on or are composed of other solutions
- Cross-reference to SDEs, solution factories, and value chains
- Search, discovery, and comparison of solutions across the platform
- Governance hooks: ownership, policy assignments, and compliance status

3. Solution Development Environment (SDE)

The Solution Development Environment (SDE) is the atomic operational unit of qala. Every developer, team, or automated process works within an SDE. SDEs are solution-agnostic by design — they define the conditions for creating, maintaining, and operating solutions, not the solutions themselves.

3.1 SDE Properties

An SDE is characterized by the following properties:

Property	Description
Deployable	Can be deployed to a platform, cloud environment, or physical machine
Configurable	All parameters, settings, and options are externally definable
Distributable	Can be shared, cloned, replicated, and distributed across teams or sites
Version-controlled	Full history of environment state, including snapshots and rollbacks
Composable	Multiple SDEs can be combined or linked within a Solution Factory
Scalable	Supports single-user and enterprise-scale workloads without model change

3.2 SDE Components

Each SDE contains the following subsystems and resources:

CONFIGURATION & IDENTITY

- Environment variables and configuration files
- Settings, parameters, and options
- User preferences and profiles

TOOLING

- Toolkits containing toolsets, which contain tools
- Tools can be linked together to form toolchains
- Third-party vendor tool integrations, managed and versioned
- IDE integrations (VSCode, JetBrains, etc.)

LANGUAGES & RUNTIMES

- Supported programming languages and versions
- Runtime environments, compilers, interpreters, and build systems

CONTENT & KNOWLEDGE

- Solution documentation, charters, design documents, files
- Solution content management system (CMS)
- Playbooks, blueprints, templates, and standards

ASSETS & ARTIFACTS

- Asset, artifact, capital, and resource repositories
- Binary and package repositories with versioning

COMMUNICATIONS & NETWORKING

- Communications and networking modules
- Connections to external services, APIs, and data sources

3.3 SDE Lifecycle States

An SDE transitions through the following lifecycle states:

State	Trigger	Description
Provisioning	SDE creation request	Environment being assembled from template or definition
Active	Provisioning complete	Fully operational; development and build activities occur
Snapshotted	User or CI action	State captured; environment continues to run
Suspended	Inactivity or admin action	Resources released; state preserved for resumption
Rolled Back	Restore request	Environment restored to a prior snapshot version
Archived	Retention policy or admin	Moved to long-term storage; retrievable on demand
Terminated	Explicit deletion	Permanently decommissioned; backups retained per policy

3.4 SDE Backup, Recovery, and Archiving

qala provides comprehensive data protection for all SDEs through three integrated subsystems:

3.4.1 Backup System

Supports scheduled and event-driven backups. Both full and incremental backup modes are available. Backups are stored in secure, versioned storage (on-premises or cloud) and linked to metadata including timestamp, environment version, and associated project.

3.4.2 Recovery & Restore System

Provides point-in-time restore, environment rollback, and artifact rehydration. User-triggered restores are available via the Access Portal or IDE integration. Restored environments are automatically validated for compatibility with current pipelines.

3.4.3 Archiving System

Moves inactive or legacy SDEs, snapshots, and artifacts to long-term storage. Enforces retention policies, supports regulatory compliance, and provides retrieval and reactivation of archived environments. AI recommendations identify candidate SDEs for archival based on usage patterns.

4. Solution Factory (SF)

A Solution Factory (SF) is a coordinated, networked collection of SDEs. The factory orchestrates the environments within it to produce solutions in a consistent, repeatable, and scalable way. qala itself is the root Solution Factory — capable of producing hierarchical child factories, each potentially scoped to a team, product line, business unit, or domain.

4.1 Factory Hierarchy

Solution Factories are arranged in a hierarchical tree rooted at the qala platform instance:

```
qala (Root SF) → Domain SF → Team SF → Project SDE → Developer SDE
```

Each level of the hierarchy inherits governance, tooling, and configuration from its parent while maintaining the ability to specialize and extend. Factories at any level can produce solutions, manage environments, and spawn child factories.

4.2 Factory Orchestration

Within a Solution Factory, SDEs are networked and orchestrated through the following mechanisms:

- Centralized workflow automation coordinating build, test, release, and deployment pipelines across SDEs
- Shared artifact and package repositories accessible to all environments within a factory
- Standardized event streaming (e.g., Kafka) enabling loose-coupled communication between services and environments
- Policy enforcement ensuring all SDEs in a factory adhere to governance, security, and quality standards
- Coordinated CI/CD pipelines spanning multiple SDEs and environments

4.3 Solution Factory Services

Every Solution Factory exposes the following core service categories to its SDEs:

Service Category	Responsibilities
Environment Management	Provisioning, cloning, teardown, monitoring, and version control of SDEs
CI/CD & Build	Hermetic build execution, pipeline orchestration, artifact generation

Service Category	Responsibilities
Testing & QA	Test case management, automated execution, defect tracking, analytics
Release Management	Versioned release packaging, approval workflows, deployment automation
Artifact & Package Mgmt	Binary storage, dependency management, 3rd-party integrations
Security & Compliance	SAST/DAST/IAST, vulnerability scanning, secrets management, audit logging
Benchmarking & Analytics	Performance, quality, and process metrics with dashboards and alerts
AI & Recommendations	Predictive insights, optimization suggestions, anomaly detection
Governance & Standards	Policies, templates, playbooks, and standards enforcement
Data Platform	Pipelines, warehouse, MDM, and analytics aggregation

5. Hermetic Build System

qala mandates hermetic build environments for all solution artifacts. Hermetic builds guarantee that the same source code and inputs will always produce the same outputs, regardless of when or where the build is executed.

5.1 Hermetic Environment Design

- All build environments are fully isolated from the host system and external network
- Environments are immutable: no state persists between builds unless explicitly captured as an artifact
- All dependencies — compilers, libraries, tools, runtimes — are frozen at declared versions
- Build environments are defined as code (Dockerfile, Nix expressions, or equivalent) and version-controlled
- Containerization (Docker) and orchestration (Kubernetes/Helm) are the primary deployment mechanisms
- Ephemeral environments are created on demand for each build and destroyed on completion

5.2 Build Attestation & Signatures

Every build artifact produced by qala is accompanied by a signed attestation record:

Attestation Field	Description
Build ID	Unique identifier for the build execution instance
Source Commit Hash	Cryptographic hash of the source code at build time
Environment Hash	Hash of the complete build environment definition
Dependency Manifest	Full lock file of all resolved dependencies with hashes
Build Timestamp	UTC timestamp of build initiation and completion
Artifact Hash	SHA-256 of each produced artifact binary
Builder Identity	Signing key of the CI/CD agent or user who triggered the build
Signature	Cryptographic signature over all attestation fields

Attestations are stored immutably alongside artifacts and are verifiable at any point in the artifact lifecycle. Deployment pipelines may be configured to reject artifacts without valid attestations.

5.3 Build Pipeline Stages

A standard hermetic build pipeline in qala consists of the following ordered stages:

1. Source Fetch — checked-out at exact commit hash, verified against SCM
2. Environment Provision — hermetic container instantiated from locked definition
3. Dependency Resolution — all deps resolved from internal mirror, hashes verified
4. Compile / Build — source compiled within the isolated environment
5. Unit Test — automated test suite executed, results captured
6. Security Scan — SAST analysis applied to source and dependencies
7. Artifact Package — output binaries packaged, hashed, and signed
8. Attestation Generation — build provenance record created and signed
9. Artifact Publication — packages stored in Artifact & Package Manager
10. Environment Teardown — hermetic container destroyed; no state persists

6. CI/CD & Configuration Management

qala integrates Continuous Integration, Continuous Deployment, and Configuration Management (CI+CD+CM) as tightly coupled, first-class platform capabilities. Every code commit, configuration change, and environment mutation flows through this system.

6.1 Continuous Integration (CI)

- Pipelines are automatically triggered on commit, pull request, or merge events from version control
- Every pipeline run is executed in a hermetic, ephemeral build environment (see Section 5)
- Code quality gates enforce linting, static analysis, test coverage thresholds, and complexity limits
- Security policies (SAST, dependency scanning) are mandatory gates; failures block artifact promotion
- Pipeline definitions are version-controlled alongside source code
- AI-driven pipeline optimization identifies bottlenecks and recommends restructuring

6.2 Continuous Deployment / Delivery (CD)

qala supports multiple deployment strategies, configurable per solution and environment tier:

Strategy	Description
Blue/Green	Two parallel environments; traffic switched atomically on validation
Canary	Progressive traffic shift to new version; automatic rollback on degradation
Rolling	Incremental instance replacement; maintains capacity throughout
Recreate	Full teardown and redeploy; acceptable for non-critical or stateless services
Feature Flag	Code deployed to all; features gated by configuration flags per user segment

6.3 Configuration Management (CM)

- All environment configurations are version-controlled and stored in a central configuration repository
- Configuration changes trigger their own pipelines: validate, apply, verify, and record
- Configuration drift is detected continuously; automatic remediation or alerts are raised
- Environment templates are parameterized by technology stack, project, and release context

- Configurations are auditable: every change is logged with author, timestamp, and justification
- Secrets are excluded from configuration files and managed through the Secrets Vault (see Section 9)

6.4 Version Control Management

qala provides a unified version control management layer that integrates with industry-standard SCM systems (Git-compatible) while adding platform-level capabilities:

- Branch policies, merge requirements, and approval workflows enforced platform-wide
- Automatic linking of commits to solutions, projects, bugs, and releases
- Semantic versioning enforced for all solution artifacts and SDE configurations
- Change impact analysis: automatically identify which solutions, tests, and deployments are affected by a given commit
- AI-assisted code review: suggestions for code quality, security, and standards compliance

6.5 Change Control Management

All significant changes to solutions, environments, or infrastructure follow a governed change control process:

- Change requests are logged with impact assessment, risk rating, and rollback plan
- Approval workflows are configurable by change type, risk level, and environment tier
- Post-implementation reviews are automatically scheduled and linked to change records
- Emergency change procedures support expedited approval for critical fixes
- All changes are traceable to the business objective, project, or incident that motivated them

7. Testing & Prototyping Management

qala treats testing as a continuous, integrated activity — not a phase. Testing is embedded into every stage of the solution lifecycle, from early prototyping through production monitoring.

7.1 Test Management

- Centralized test case, plan, suite, and result management across all solutions and environments
- Test cases are version-controlled and linked to requirements, user stories, and solution components
- AI-driven test case generation synthesizes test scenarios from specifications and historical defect data
- AI-driven prioritization ranks test cases by risk, coverage gap, and historical failure rate
- Test results feed into the Benchmarking and Analytics services automatically

Test Levels Supported

Level	Scope
Unit	Individual functions, classes, and modules
Integration	Inter-component and inter-service interactions
System	End-to-end behavior of complete solutions
Performance	Throughput, latency, and resource utilization under load
Security	SAST, DAST, IAST, penetration testing
Acceptance	Validation against business requirements and user expectations
Regression	Automated re-verification after changes

7.2 Defect Tracking

- Bugs and defects are logged, categorized by severity and priority, and tracked through resolution
- Defects are automatically linked to the build, commit, release, and project that introduced them
- Defect density and MTTR (Mean Time to Resolution) metrics feed into the Benchmarking service
- Predictive defect detection uses historical data to flag high-risk code areas before release
- Self-healing test capabilities detect and repair brittle tests in dynamic environments

7.3 Prototyping & Sandbox Management

- Automated provisioning of isolated sandbox environments for proof-of-concept (POC) work
- Prototypes are linked to parent projects, solutions, and features in the registry
- Prototype feedback is captured and analyzed automatically; AI summarizes findings and recommends next steps
- Sandbox environments are ephemeral by default; promotion to persistent environments requires governance review
- Resource quotas and time limits are enforced on sandbox environments to control cost

7.4 Test Environments

Environment provisioning for testing is fully automated and integrated with the CI/CD pipeline:

- On-demand provisioning of test environments from version-controlled templates
- Environments are torn down automatically after test execution to reclaim resources
- Environment cloning allows parallel test runs across environment configurations
- Release management gates require defined test suites to pass before deployment approval

8. Benchmarking & Performance Analysis

qala integrates multi-dimensional benchmarking across performance, quality, and process dimensions. Benchmarking data is collected continuously and aggregated into dashboards, alerts, and AI-driven recommendations.

8.1 Performance Benchmarking

- CPU, memory, disk I/O, and network utilization per solution, component, and environment
- Response time and throughput under nominal and stress load conditions
- Scalability profiling: behavior under progressive load increases
- Automated performance tests executed in CI/CD on every release candidate
- Cross-release and cross-environment comparison to identify regressions

8.2 Quality Benchmarking

- Code complexity, maintainability index, and duplication metrics
- Linting and coding standard adherence scores
- Test coverage: unit, integration, and system levels
- Defect density, severity distribution, and MTTR trends
- Technical debt tracking and amortization planning

8.3 Process Benchmarking

- Development velocity: story points or equivalent per sprint or iteration
- Deployment frequency and lead time from commit to production
- Release success rate, rollback frequency, and hotfix rate
- CI/CD pipeline duration and failure rates by stage

8.4 Cross-Solution Benchmarking

- Compare metrics across projects, teams, applications, and products within a factory
- Identify high-performing patterns and recommend adoption in lower-performing areas
- AI-driven benchmarking predicts potential performance or defect hotspots before they materialize
- Scenario benchmarking simulates peak load, environment-specific conditions, or projected growth

8.5 Reporting & Alerting

Audience	Dashboard Content
Executives	KPI trends, release velocity, quality posture, risk summary
Architects	Code quality, technical debt, dependency health, system complexity
QA & Release Mgrs	Test coverage, defect rates, pipeline health, deployment outcomes
Developers	Personal SDE metrics, build times, code quality feedback, test results

9. Security & Privacy Management

Security and privacy are embedded into every layer of the qala platform — not appended as afterthoughts. Every SDE, pipeline, artifact, and data flow is subject to continuous security monitoring and policy enforcement.

9.1 Security Testing Integration

Type	Acronym	Integration Point
Static Analysis	SAST	Pre-commit hooks and CI pipeline build stage
Dynamic Analysis	DAST	Post-deploy automated scanning in staging environments
Interactive Analysis	IAST	Instrumented test execution across integration test stage
Dependency Scan	SCA	Dependency resolution stage; blocks known-vulnerable packages
Container Scan	—	Base image and runtime container scanning before promotion
Penetration Testing	—	Managed via test management service; linked to releases

9.2 Vulnerability Management

- Continuous vulnerability scanning of code, dependencies, containers, and infrastructure
- Risk scoring by severity, exploitability, and asset criticality
- Automated patching recommendations with impact analysis before application
- Remediation tracking: vulnerabilities are linked to tickets, owners, and SLA targets
- High-severity findings trigger immediate alerts and can block pipeline promotion

9.3 Threat Modeling & Risk Assessment

- Threat models are created and versioned per application and solution during design phase
- Attack surface mapping is updated automatically as solution structure changes
- Risks are assigned severity, probability, and mitigation plans
- Threat intelligence feeds keep the platform current with emerging attack patterns

9.4 Privacy Management

- PII identification and classification across all data flows and stores
- Data masking, anonymization, and encryption policies enforced per data classification
- Retention and deletion policies with automated enforcement
- Audit logs of all access to and modification of sensitive data
- Compliance dashboards for GDPR, CCPA, ISO 27001, SOC 2, and configurable regulations

9.5 Secrets Management

- All credentials, API keys, certificates, and secrets stored in a hardware-backed vault
- Secrets are never embedded in source code, configuration files, or environment variables
- Dynamic secret generation with short TTLs for ephemeral environments
- Access to secrets is audited, role-gated, and revocable in real time

9.6 Access Control & Audit

- Role-Based Access Control (RBAC) across all platform modules and resources
- Multi-Factor Authentication (MFA) required for all human access
- Single Sign-On (SSO) integration with enterprise identity providers
- Full audit trail for deployments, environment changes, and sensitive data access
- Cross-SDE containment: security incidents in one SDE cannot propagate to others

10. Release & Deployment Management

qala manages the complete release lifecycle from packaging through production deployment and post-release monitoring. Release management is tightly integrated with testing, security, and governance to ensure only validated, compliant artifacts reach production.

10.1 Artifact & Package Management

- Centralized repository for all compiled outputs: libraries, binaries, containers, packages, and artifacts
- Semantic versioning enforced on all artifacts with full dependency graph tracking
- Artifacts are immutable after publication; updates always produce a new versioned artifact
- Third-party vendor packages are proxied through an internal mirror for integrity verification and availability
- Artifact promotion gates: artifacts must pass security and quality checks before promotion to higher environment tiers
- Rollback is achieved by redeploying a prior artifact version — no recompilation required

10.2 Release Pipeline

A standard release pipeline in qala progresses through the following stages:

Build → Unit Test → Security Scan → Package → Integration Test → Performance Test → Staging Deploy → Acceptance Test → Production Deploy → Post-Deploy Verification

Each stage has configurable pass/fail criteria. Failed stages halt progression and trigger notifications. Stages may run in parallel where dependencies allow.

10.3 Environment Tiers

Tier	Purpose	Promotion Gate
Development	Active development and local testing	Unit tests passing
Integration	Cross-service and cross-component testing	Integration test suite passing

Tier	Purpose	Promotion Gate
Staging	Production-equivalent pre-release validation	Performance + acceptance tests
Production	Live environment serving end users	Release manager approval + all gates

11. Governance, Standards & Lifecycle Management

qala embeds governance as a native platform capability. Every solution, environment, and artifact exists within a governed context that defines ownership, policies, standards, and lifecycle rules.

11.1 Governance Model

- Every solution has a designated owner and contributing team with defined roles and responsibilities
- Governance policies are defined at the factory level and inherited by all child environments and solutions
- Policy-as-code: governance rules are machine-readable, version-controlled, and automatically enforced
- Exceptions to policies are tracked, justified, time-limited, and subject to review

11.2 Solution Playbooks & Plans

Every solution in qala may have associated playbooks that capture its design intent, operational procedures, and lifecycle plans:

- Playbooks: step-by-step operational procedures for deployment, incident response, and scaling
- Blueprints: architectural designs and component diagrams
- Roadmaps: multi-horizon feature and capability plans
- Timelines and Gantt charts: schedule representations with dependencies
- Solution Books: the complete documentation repository for a given solution

11.3 SDE Governance & Lifecycle State Management

SDEs are governed through their full lifecycle with state transitions subject to policy validation:

- Templates define approved SDE configurations; deviations require justification and approval
- Lifecycle state transitions (e.g., Active → Archived) are audited and may require multi-party approval
- Resource quotas and cost allocations are tracked per SDE and reported to owners
- Compliance scanning verifies SDE configurations against security baselines on a scheduled basis

11.4 Standards, Templates & Playbooks Library

- Central library of approved project templates, environment definitions, pipeline configurations, and code scaffolds
- Templates are versioned, categorized by technology stack and solution type, and recommended by AI based on context
- Deviations from standard templates are tracked as technical debt items
- Standards are reviewed on a defined cadence and updated to reflect evolving best practices

12. Solution Value Chain

qala provides a Solution Value Chain model that maps the end-to-end flow of value from inputs (resources, ideas, requirements) through to outcomes (deployed solutions, user value, business results). The value chain is tracked, measured, and optimized continuously.

12.1 Value Chain Components

Component	Description
Inputs	Requirements, ideas, user feedback, market signals, resources
Design	Solution modeling, architecture, and specification
Build	Development, configuration, and integration within SDEs
Verify	Testing, security scanning, benchmarking, and quality validation
Release	Packaging, signing, and artifact promotion
Deploy	Environment provisioning and solution deployment
Operate	Monitoring, incident management, and continuous optimization
Outcomes	User value delivered, business objectives achieved, metrics realized

12.2 Resource Management

qala tracks all resources consumed across the value chain:

- Human resources: developer time, team assignments, and capacity planning
- Compute resources: CPU, memory, storage, and network per SDE and pipeline run
- Financial resources: cost allocation per project, team, solution, and environment tier
- Third-party resources: licensed tools, vendor services, and external dependencies
- AI-driven resource optimization recommends reallocation to reduce waste and improve throughput

12.3 Solution Ownership System

Every solution artifact in qala has a clearly defined ownership model:

- Solutions have a primary owner (person or team) responsible for quality, compliance, and roadmap
- Contributing owners may be defined for shared responsibilities (security, performance, documentation)
- Ownership transfers are tracked and require formal acknowledgment

- Orphaned solutions (no active owner) are flagged and escalated to factory administrators

13. Data Platform & AI Integration

qala is powered by a unified data platform that aggregates telemetry, metrics, logs, and events from every platform component. AI agents operate continuously against this data to provide intelligence across all platform functions.

13.1 Data Platform Architecture

- Event streaming backbone (Apache Kafka or equivalent) connecting all microservices
- Data pipelines for ETL, transformation, and aggregation of platform telemetry
- Central data warehouse for historical analytics and trend analysis
- Master Data Management (MDM) ensuring consistent reference data across services
- Data lineage tracking: every data point traces to its origin event and transformation history

13.2 AI Capabilities

AI Capability	Application
SDE Optimization	Recommends configuration improvements for performance and cost efficiency
Pipeline Bottleneck Detection	Identifies slow stages and recommends parallelization or caching
Predictive Defect Detection	Flags high-risk code areas using historical defect patterns
Test Case Generation	Synthesizes test scenarios from specifications and past failures
Anomaly Detection	Detects unusual patterns in metrics, logs, and security events
Resource Prediction	Forecasts compute and capacity needs based on activity trends
Content Suggestions	Recommends documentation structure, template selection, and standards
Security Threat Analysis	Correlates security events to identify attack patterns and risks
Backup Schedule Optimization	Recommends backup frequency based on change velocity and risk
Prototype Feedback Analysis	Synthesizes prototype results into actionable design recommendations

14. API & Microservice Architecture

qala is implemented as a collection of independently deployable microservices, each owning its data and exposing a versioned API. Services communicate asynchronously via event streams and synchronously via REST or gRPC where required.

14.1 Core Microservices

Service	Responsibilities	Key Events
User & Identity	Users, RBAC, MFA, audit logging	USER_EVENTS
SDE Management	Provisioning, snapshots, rollbacks, lifecycle	SDE_EVENTS
Workspace & CMS	Workspaces, content, versioning, collaboration	CMS_EVENTS
Workflow & CI/CD	Pipelines, builds, hermetic environments	BUILD_EVENTS
Artifact & Package	Binary storage, versioning, 3rd-party tools	ARTIFACT_EVENTS
Data Platform	Pipelines, warehouse, MDM, analytics	DATA_EVENTS
Security & SEM	Threat detection, secrets vault, policies	SECURITY_EVENTS
AI Agents	Recommendations, predictions, context hooks	AI_RECOMMENDATIONS
Notifications	Alerts, portal messages, email/SMS/chat	NOTIFICATIONS
Platform Admin	Templates, policies, configurations, governance	—

14.2 API Gateway

All external and internal API traffic routes through a central API Gateway that provides:

- Authentication (JWT/OAuth 2.0) and Authorization (RBAC policy enforcement)
- Request routing to the appropriate microservice
- Rate limiting, caching, and request/response transformation
- TLS termination and mutual TLS for service-to-service calls
- API versioning and backward compatibility management
- Observability: request tracing, latency metrics, and error rate monitoring

14.3 Key API Endpoints

Endpoint	Method	Description
/auth/login	POST	Authenticate user, return JWT
/users/{id}	GET/PUT/DELETE	User management
/sdes	GET/POST	List or create SDEs
/sdes/{id}/snapshot	POST	Capture SDE state snapshot
/sdes/{id}/rollback	POST	Restore SDE to prior snapshot
/sdes/{id}/clone	POST	Clone SDE to new instance
/ci/pipelines/{id}/run	POST	Trigger CI/CD pipeline execution
/artifacts/{id}	GET/DELETE	Manage build artifacts
/security/threats	GET	List active threats and findings
/security/secrets	GET/POST	Access or store secrets in vault
/analytics/predict	POST	Submit predictive analytics request
/bugs	GET/POST	List or report defects

15. Deployment Architecture

qala is designed for cloud-native deployment with full support for on-premises and hybrid topologies. The platform is containerized throughout, orchestrated via Kubernetes, and configured through declarative infrastructure-as-code.

15.1 Infrastructure Stack

Layer	Technology
Containerization	Docker — all services and build environments packaged as containers
Orchestration	Kubernetes with Helm charts for service deployment and configuration
Service Mesh	Istio or Linkerd for mTLS, traffic management, and observability
Event Streaming	Apache Kafka for platform-wide event bus
Secret Storage	HashiCorp Vault or cloud-native KMS
Artifact Storage	OCI-compatible registry + object storage (S3-compatible)
Observability	OpenTelemetry, Prometheus, Grafana, and centralized log aggregation
IaC	Terraform / Pulumi for infrastructure provisioning

15.2 Multi-Tenancy & Isolation

- Each Solution Factory operates in a logically isolated namespace within the platform
- Resource quotas are enforced at the factory, team, and SDE level
- Network policies prevent cross-tenant data access at the infrastructure layer
- Encryption at rest and in transit is mandatory for all tenant data
- Audit logs are immutable and isolated per tenant for compliance purposes

15.3 Scalability

- All microservices are horizontally scalable; no stateful singletons in the critical path
- Auto-scaling policies based on CPU, memory, and custom application metrics
- Build environments are ephemeral and scale to zero when idle
- Event streaming throughput scales with Kafka partition count and consumer groups
- AI inference services scale independently of core platform services

Appendix A: Glossary

Term	Definition
SDE	Solution Development Environment — the atomic operational unit in qala, providing all tools, configuration, and resources needed to create and manage a solution
SF	Solution Factory — a coordinated collection of networked SDEs producing solutions in a consistent, repeatable way
SFOS	Solution Factory Operating System — the qala platform itself, providing the operating model for all factories and environments
CI	Continuous Integration — automated build and test on every code change
CD	Continuous Deployment/Delivery — automated promotion of validated artifacts to target environments
CM	Configuration Management — governed control of all environment and system configuration
SAST	Static Application Security Testing — code analysis without execution
DAST	Dynamic Application Security Testing — testing against a running application
IAST	Interactive Application Security Testing — instrumented testing during execution
RBAC	Role-Based Access Control — access permissions determined by assigned roles
MDM	Master Data Management — ensuring consistency of shared reference data
MTTR	Mean Time to Resolution — average time from defect discovery to resolution
PII	Personally Identifiable Information — data that can identify a natural person
IaC	Infrastructure as Code — infrastructure provisioning defined and managed as version-controlled code
POC	Proof of Concept — a prototype used to validate feasibility of an approach

Appendix B: Event Stream Topics

Topic	Published By
USER_EVENTS	User & Identity Service
SDE_EVENTS	SDE Management Service

Topic	Published By
CMS_EVENTS	Workspace & CMS Service
BUILD_EVENTS	Workflow & CI/CD Service
ARTIFACT_EVENTS	Artifact & Package Management Service
DATA_EVENTS	Data Platform Service
SECURITY_EVENTS	Security & SEM Service
NOTIFICATIONS	Notifications Service
AI_RECOMMENDATIONS	AI Agents Service